Plausibility Masks Executability Failures in LLM-Based Planning

Mei Si
Department of Cognitive Science, Rensselaer Polytechnic Institute, Troy, New York, USA

Abstract

Language-model plans can read coherently while failing at an action whose prerequisites do not hold. We trace this gap across three linked analyses: an audit of a January 2026 planning study, a September rerun of its original Logistics tasks, and a September evaluation on new, longer Logistics tasks. The audit shows how task reachability, operator signatures, and the success criterion each change the interpretation of a planning benchmark. On the original Logistics tasks, the later models and explicit action signatures produce 139 valid final plans in 140 runs, a marked improvement that cannot be attributed to one change in isolation. Yet on four new, solvable tasks with more objects and longer plans, precondition failures recur, especially when a package or truck is assumed to be at the location named in a `load` action. The new cohort yields 126 valid final plans in 140 runs after correcting four final-plan extraction artifacts; 13 genuine precondition failures and one unmet-goal ending remain. These observations narrow the claim: failure on the old tasks is not stable across model and prompt revisions, but fluent generation still requires exact checks of state, action applicability, and the complete final sequence.

1. Introduction

Language models can produce fluent action sequences that fail when checked against explicit states and preconditions. This distinction motivates symbolic execution as an evaluation of planning rather than a judgment of textual coherence (Valmeekam et al., 2023). A planning benchmark needs three checks before its success rates can be interpreted: whether each task is reachable, whether action arguments match the executable operator, and whether the entire returned sequence is valid.

We revisit the January 2026 outputs of a four-domain, five-prompt study. Its original interpretation emphasized *symbolic action collapse*: an apparently sensible plan omitted steps needed to satisfy later preconditions. The saved executor records show genuine precondition failures, but the original aggregate also includes five unreachable tasks among sixteen. The manuscript misstated the coded Logistics argument order; the actual prompts exposed numbered variables and their preconditions without naming the argument slots. Ten runs reached a goal and then proposed an invalid action. Correcting these three points changes what the outcome counts mean, while preserving the observed precondition failures.

The paper follows the failure pattern across time and task demands. First, we audit the January benchmark and outcomes, including reachability and complete-sequence execution. Second, we repeat the original Logistics tasks in September with explicitly named action arguments and find that the earlier failures largely disappear. Third, we examine new solvable Logistics tasks with longer

action dependencies and find the same kinds of missing location prerequisites in the completed raw responses. This progression distinguishes an outdated success rate from a continuing executability risk. The January archive lacks raw model responses, so its diagnostic predicates cannot establish exactly what the models intended; the September examples use saved final plans.

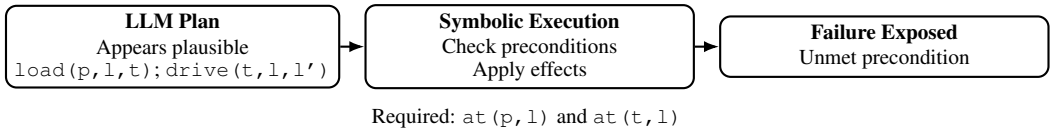


Figure 1: **Plausibility masks executability failure.** A plan can read as reasonable while omitting required intermediate actions or state changes; exact symbolic execution exposes the hidden failure as an unmet precondition.

2. Related Work

Direct language-model planning has been studied in embodied settings, including zero-shot action generation and few-shot grounded planning (Huang et al., 2022; Song et al., 2023). Generalized planning work instead asks models to synthesize programs that solve new tasks in a PDDL domain (Silver et al., 2024). Chain-of-thought prompting encourages intermediate reasoning (Wei et al., 2022); Tree of Thoughts explores alternative reasoning paths (Yao et al., 2023a). These are different settings from our one-response, exactly checked action sequences. PlanBench already establishes that language-model performance on symbolic action and change reasoning can fall short (Valmeekam et al., 2023); Natural Plan tests trip, meeting, and calendar tasks and reports declining success on more complex instances (Zheng et al., 2024). A planning survey also distinguishes direct plan generation from formalization and hybrid approaches (Aghzal et al., 2025). Our contribution is narrower: we audit reachability, positional action signatures, first missing predicates, final-plan extraction, and whether every returned action executes. We do not claim to discover precondition failure as a new phenomenon.

Hybrid approaches assign different roles to language models and formal planning. LLM+P translates requests into planning problems for a solver (Liu et al., 2023), and LLM-Modulo combines model proposals with external verification (Kambhampati et al., 2024). Work on planning formalizers surveys the construction of formal action models (Tantakoun et al., 2025); Proc2PDDL evaluates deriving action parameters, preconditions, and effects from procedural text (Zhang et al., 2024). These approaches make both verification and the quality of the formal domain important. Our January audit illustrates why a benchmark's own reachability and operator order must be checked before interpreting success rates or attributing a rejected action to model reasoning.

Interactive systems add feedback or grounding to action selection. ReAct interleaves reasoning and acting (Yao et al., 2023b); Inner Monologue uses environmental feedback in embodied tasks (Huang et al., 2023); DEPS uses execution descriptions and explanations to correct plans (Wang et al., 2023); and SayCan grounds language proposals in robotic affordances (Ahn et al., 2022). Our offline experiment cannot evaluate such feedback-based repair. Instead, it identifies the preconditions a generated final plan fails when applied to an explicit state, motivating precise checks in both direct-planning evaluations and interactive systems.

3. Problem Setting and Evaluation Framework

We represent a task as an initial predicate set s_0, deterministic action schemas $\mathcal{A}$ with conjunctive preconditions and add/delete effects, and a conjunctive goal G (Ghallab et al., 2004; McDermott et al., 1998). An action is applicable when all grounded preconditions hold; applying it deletes and adds the specified predicates. A task is *solvable* if some applicable action sequence reaches a state containing G. We enumerate reachable states from s_0 using the supplied schemas to check this property before interpreting a model failure.

The five prompt conditions are `free_form`, `restricted_actions`, and three restricted variants requesting hidden chain-of-thought, explicit state reasoning, or verbose reasoning. All request a numbered final list of grounded actions. The executor parses that list, checks preconditions in sequence, and records executed steps, goal attainment, and the first failure reason. For the full-sequence outcome, *full-plan success* requires the goal flag, `failure_reason=ok`, and zero unknown-action, precondition-failure, and multi-action counts. This is stricter than the January summary's goal flag alone: the implementation can retain an earlier goal attainment despite a later failed action.

We report counts and descriptive fractions for the fixed January outputs. Temperature settings are pooled within each model (two settings except GPT-5.2, which has one), so these are run-level summaries, not matched cross-model estimates. A precondition record diagnoses the executor's state and interface; it does not, by itself, prove why the model selected that action.

4. Experimental Setting

The archive contains 560 cleaned runs: 16 instances (four each in Blocks World, Keys-and-Doors, Logistics, and Tools), five prompt conditions, and four models. GPT-4.1, Gemini 2.5 Flash, and Gemini 3 Flash each have two temperature settings (160 runs per model); GPT-5.2 has one (80 runs). The original CSV archive records January 18–19, 2026 file dates. Two September Logistics evaluations provide a linked follow-up: a rerun of the same four instances and a set of four new, solvable problems with longer witness plans. Both state the package–truck–location action signature explicitly. We report each cohort separately because the tasks, prompt specification, and model date differ from January.

We checked reachability by forward exploration of grounded operators from each initial state until the goal was reached or no new states remained. Table 1 gives the outcome. In `keys_1` and `keys_2`, accessing room r3 requires `passable(r3)`, but the available `unlock` action adds `passable(r2)`. In `tools_2`–`tools_4`, the available actions cannot establish the needed actor/object locations or heating prerequisites; the Tools domain has no movement action. These five tasks have no path to their goals under the coded schemas. The other eleven are reachable. Each instance contributes 35 runs, hence 175/560 runs were assigned impossible tasks.

The coded Logistics schemas take `load(package, location, truck)` and `unload(package, location, truck)`. The earlier manuscript's package–truck–location order was a documentation error, corrected here. The actual prompts showed `load(...)` and the variable-based preconditions/effects: `at(?x0,?x1)` and `at(?x2,?x1)` imply that the location is the second argument, while `in(?x0,?x2)` identifies the third as the truck. Thus the requirements were recov-

Domain	Solvable	Unsolvable	Unsolvable IDs
Blocks World	4	0	—
Keys-and-Doors	2	2	`keys_1`, `keys_2`
Logistics	4	0	—
Tools	1	3	`tools_2`–`tools_4`

Table 1: Reachability under the supplied January action schemas.

erable from the prompt, although the signature was not stated in words. We evaluate each proposal against that coded order and avoid claiming that a particular failure proves the model's intended binding. Full January response text was not retained in the supplied archive; the examples below come from executor records.

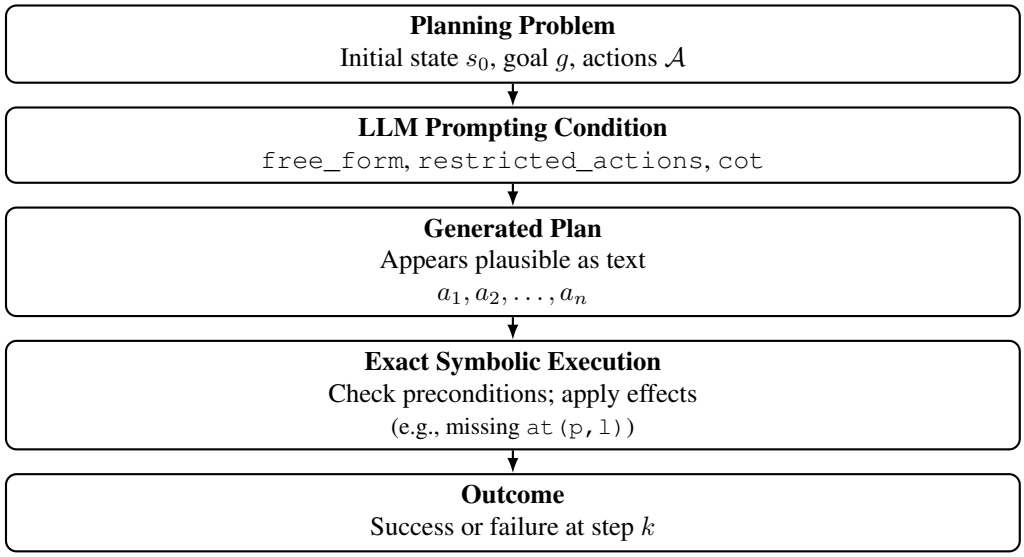


Figure 2: Experimental pipeline. Generated action sequences are checked against exact symbolic semantics; the audit also checks task reachability and the operator interface.

5. Results and Discussion

5.1 January audit: corrected success and prompt comparison

The January summary counted 292/560 (52.1%) runs whose executed prefix reached the goal. Of these, 282/560 (50.4%) returned fully executable goal-reaching sequences; the remaining ten GPT-4.1 runs proposed another action after reaching the goal, which failed a precondition. The other 268/560 never reached the goal (237 precondition failures, 15 unknown actions, ten goal-not-achieved endings, and six parse failures). We count the ten post-goal errors as full-plan failures and report their timing separately. On the eleven solvable instances the same 282 successes are 282/385 (73.2%). The five unreachable tasks contributed zero successes, yet their generated sequences re-

main informative about attempts that assume unestablished predicates: 150/175 first failed on preconditions, 15 used unknown actions, seven ended without achieving the goal, and three failed to parse. Because prompts required a plan, these logs cannot establish whether a model would recognize impossibility if explicitly allowed to report no solution. Figure 3 shows both denominators as grouped bars. Free-form succeeds in 55/112 runs and restricted actions in 57/112; none of the five conditions establishes a general free-form advantage across these tasks. The unchanging low values for some domain/model combinations are partly a ceiling imposed by impossible instances, not a feature of prompting alone.

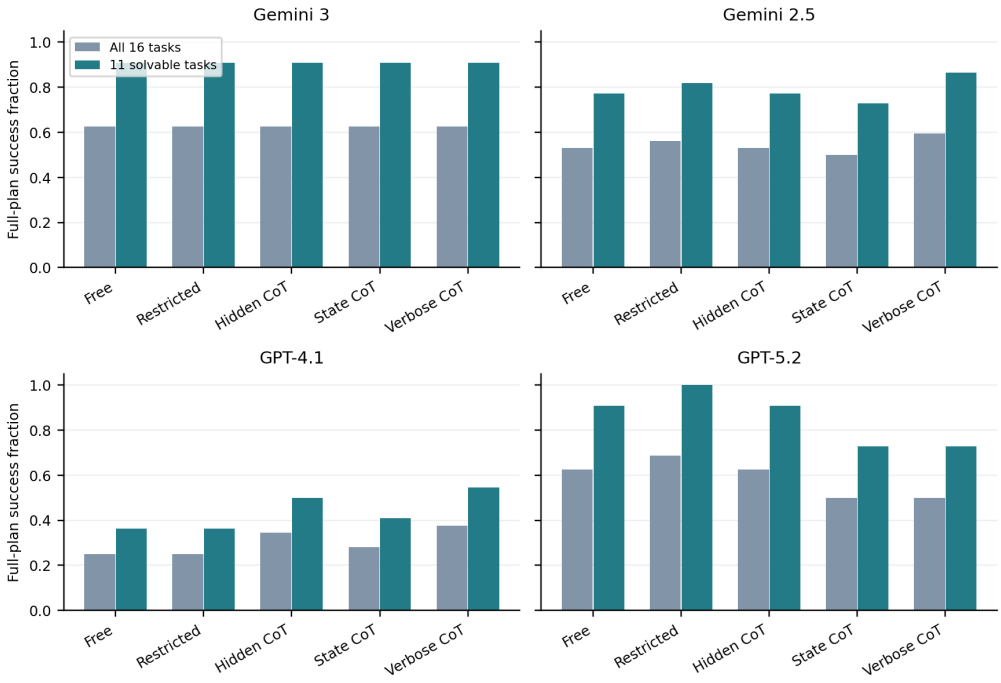


Figure 3: Full-plan success by model and prompt. Each panel compares all 16 tasks with the 11 reachable tasks; per-condition denominators are 32 versus 22 for each two-temperature model, and 16 versus 11 for GPT-5.2. Gemini 3's equal rates across prompts reflect the saved outputs, including five impossible Keys/Tools tasks in each 16-instance condition.

5.2 First failure as a diagnostic

Figure 4 counts actions executed before the first precondition failure on solvable tasks, separating 87 failures without goal attainment from ten failures after the goal was reached. The horizontal axis uses whole action counts. For example, a saved GPT-4.1 `blocks_1` free-form run executes three actions and then rejects `stack` because `holding(a)` is absent. The corresponding restricted-actions run has the same logged failure. This is an actual executor diagnostic, not a reconstruction of the missing raw response or a demonstration that free-form was better. The failure identifies an unmet state condition, but its causal origin cannot be inferred from this row alone.

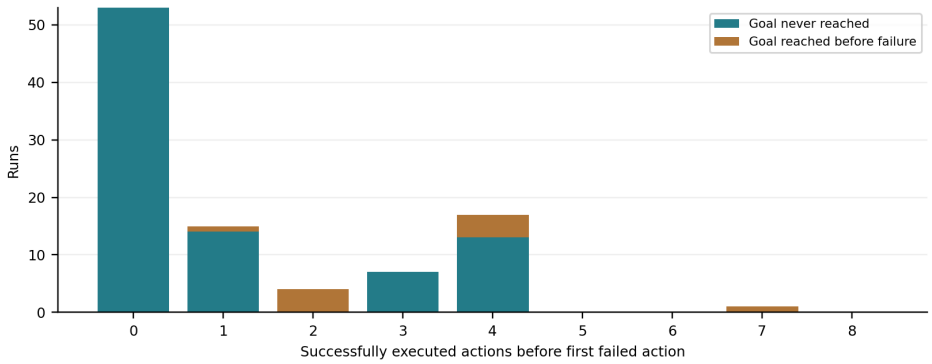


Figure 4: Integer failure depth for runs with a first precondition failure on reachable tasks, pooled across models, prompts, and temperatures. Depth zero means the first proposed action failed. Fully executable runs and unsolvable tasks are excluded. Color distinguishes failures before reaching the goal from the ten failures after reaching it.

5.3 Logistics preconditions and action bindings

All four Logistics tasks are reachable, and strict success is 3/40 for GPT-4.1, 15/20 for GPT-5.2, 28/40 for Gemini 2.5 Flash, and 40/40 for Gemini 3 Flash. Figure 5 plots the recorded precondition-failure fraction by condition; Gemini 3 remains at zero, which is a meaningful flat series. Figure 6 pools all models and temperature settings within each domain: 131/140 Blocks World runs and 86/140 Logistics runs succeed. The Logistics gaps include genuine recorded failures of the coded preconditions. The prompts gave numbered variables but no named signature, so the logs do not reveal whether a model misread argument positions or failed to track co-location.

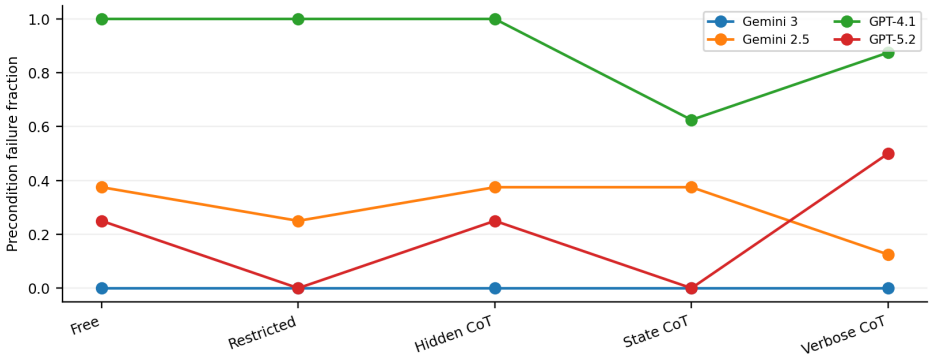


Figure 5: Fraction of Logistics runs with a recorded precondition failure by model and prompt; each point pools temperatures where present (8 runs, or 4 for GPT-5.2). Gemini 3's zero line corresponds to no such failures; load/unload use package–location–truck order.

Among Logistics precondition failures, the first failed action is usually `load` or `drive` (Figure 7). Gemini 3 does not appear in this failure-only chart because it has no Logistics failures, not

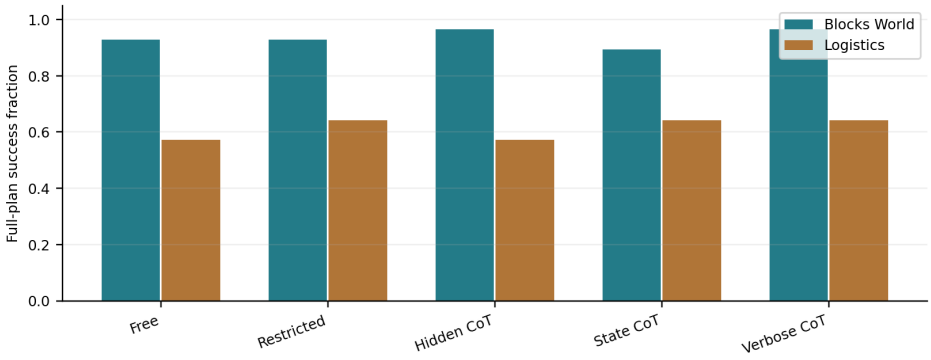


Figure 6: Full-plan success in Blocks World and Logistics, pooled over all four models and their available temperatures. Each domain has 28 runs per prompt (140 total); these are run-weighted summaries. The positional schema was shown through numbered variables, without named arguments.

because it was omitted from evaluation. Figure 8 lists the exact grounded missing predicates. In particular, `at(p1,t1)` or `at(l1,t1)` in a `load` failure is consistent with placing the conventional truck or location argument into the wrong slot. Such records expose a failed grounded precondition. The action order is derivable from the supplied variables; without the January responses, we cannot determine whether the model misbound those variables or assumed an unavailable location fact.

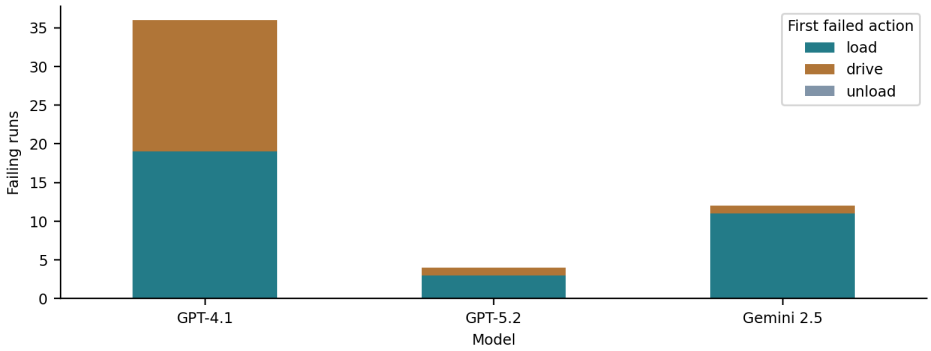


Figure 7: First precondition-failing Logistics action, by model, pooled over prompts and temperatures. Only failing runs are shown; Gemini 3 has zero and therefore no bar.

5.4 September on the original Logistics tasks: apparent recovery

To test whether the January Logistics failures persist, the separately archived September 2026 run repeats the four original instances under all five prompt conditions and four model configurations (140 calls). It names the arguments explicitly: `load(p,t,l)` and `unload(p,t,l)` mean package, truck, location; `drive(t,from,to)` has the corresponding vehicle and endpoints. The ex-

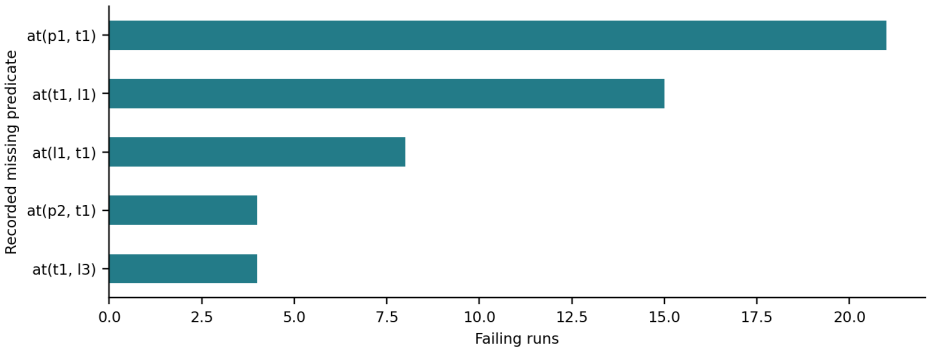


Figure 8: Exact missing predicates for first Logistics precondition failures, pooled across models, prompts, and temperatures. These strings come from the January executor logs. Load uses package–location–truck order, so a missing `at(p1,t1)` is consistent with placing a truck in the location slot.

Model	January	September logged	September final plan
GPT-4.1	3/40	37/40	40/40
GPT-5.2	15/20	19/20	19/20
Gemini 2.5 Flash	28/40	40/40	40/40
Gemini 3 Flash	40/40	40/40	40/40

Table 2: Independent Logistics cohorts. Three logged GPT-4.1 failures are parser artifacts: Markdown headings before valid `FINAL PLAN` blocks were not recognized, so numbered reasoning was evaluated instead. Re-executing the final plans gives 139/140 September successes.

ecutor uses this same order. Table 2 contrasts January full-plan success with this later cohort; it is descriptive, not an intervention isolating argument order. Model versions may also have changed.

The one genuine September failure is a concrete unmet location predicate. In GPT-5.2's `log_1` restricted-actions response, the first six actions leave `p1` at `l2`, `p2` at `l1`, and truck `t1` at `l1`. The next action is `load(p1,t1,l2)`. It satisfies `at(p1,l2)` but fails `at(t1,l2)`; a drive back to `l2` is needed first. The same model's free-form response on this exact task delivers `p1` directly to `l3`, then drives to `l2`, loads `p2`, and delivers it to `l1`, forming a valid seven-action plan. This actual paired example illustrates how two prompt outputs differ, without implying an aggregate free-form advantage.

The older Logistics tasks thus cease to be a reliable source of frequent failures in September. One genuine `load` error remains, and three false failures demonstrate the importance of extracting the final plan correctly. Because the signature, prompt wording, model date, and parsing procedure changed together, the rerun does not identify which change caused the improvement. We next ask whether the same failure pattern appears when plans require more state tracking.

Model	Simpler September	Complex logged	Complex final plan
GPT-4.1	40/40	25/40	29/40
GPT-5.2	19/20	17/20	17/20
Gemini 2.5 Flash	40/40	40/40	40/40
Gemini 3 Flash	40/40	40/40	40/40
Total	139/140	122/140	126/140

Table 3: Original versus new September Logistics tasks. Four GPT-4.1 final plans in the new cohort are valid despite strict-extractor parse errors. Independent execution of the Gemini 3 raw final plans confirms its 40/40. All 140 new-task responses are now included in the final-plan total.

5.5 September on new Logistics tasks: failures reappear

We then tested four new, solvable Logistics problems using the explicit September signatures, two or three trucks, three to five packages, and witness plans of 11–16 actions. They use the same five prompt conditions and model temperature configurations as the simpler September follow-up, but are different tasks rather than paired reruns. On September 25, 2026 (New York time), the archived batches contained all 40 GPT-4.1, 20 GPT-5.2, and 40 Gemini 2.5 Flash outputs. The Gemini 3 Flash batch subsequently completed on September 25, 2026 (New York time); independent execution of its 40 raw final plans confirms 40/40 full-plan successes. The complete new-task cohort contains 140 runs.

Across the 140 independently checked complex runs, 14 genuine failures remain after checking final plans: 11 GPT-4.1 precondition failures, two GPT-5.2 precondition failures, and one GPT-5.2 sequence that executes but ends without all goals. Ten GPT-4.1 failures are at `load` and one at `unload`; both GPT-5.2 precondition failures are at `load`. At the first failing action, six lack a truck-location predicate, six lack a package-location predicate, and one lacks an `in(package,truck)` predicate. None of these 14 failures reached the goal before the diagnosed error or termination. GPT-4.1 success declines from 40/40 on the simpler September tasks to 29/40 here; GPT-5.2 declines from 19/20 to 17/20, while Gemini 2.5 remains 40/40. These descriptive differences do not isolate task length from object count, task design, or stochastic generation.

For a concrete state-tracking example, a GPT-5.2 free-form plan for `complex_3` moves truck `t1` from `l1` to `l5`, unloads `p1`, and later proposes `load(p4,t1,l1)`. Package `p4` is at `l1`, but `at(t1,l1)` is false; the truck has not returned from `l5`. This is a genuine failure at action 17, after 16 applicable actions. Another GPT-5.2 plan for `complex_4` executes 22 actions without error but leaves a goal fact unsatisfied. The new tasks therefore restore a diagnostic contrast: success on familiar short tasks does not guarantee reliable tracking of package and truck locations in longer plans. The errors echo January's missing-location predicates and the one simpler September failure, while also distinguishing an incomplete goal from an inapplicable action. The January positional-binding ambiguity does not apply to these complex prompts because their arguments are named explicitly.

5.6 Implications for evaluation

The three cohorts support a practical evaluation procedure. The January audit changes the denominator and outcome definition; the September rerun shows that the earlier frequency of Logistics failures is not stable; the new tasks show that exact state errors can recur when dependencies grow. Benchmark builders can apply four checks before drawing a prompting or model comparison:

1. *Check task reachability.* Solve or enumerate each instance under the exact operators given to the model. Report solvable and unsolvable instances separately. An impossible instance can test whether a system recognizes impossibility, but cannot measure plan success when every response is required to be a plan.
2. *Specify and test argument bindings.* Give named, typed action signatures in the prompt and apply the same order in the executor. Preserve the actual prompt so a failed predicate can be interpreted against what the model saw.
3. *Extract and execute the final plan.* Preserve the raw response, identify the final action list, then check every proposed action and the final goal state. Report parsing errors separately from precondition violations and missing goals.
4. *Record when failure occurs.* Distinguish plans that never reach the goal from plans that reach it and then continue with invalid actions. A goal-reaching prefix is not a valid full returned sequence.

The `load` examples show why these checks matter together: a fluent step requires the package and truck to share the stated location. An exact executor identifies which location fact is absent. The result is a reproducible diagnosis of the returned plan, rather than a claim about the model's private reasoning.

6. Limitations

The primary audit covers 16 small fixed problems and archived January 2026 generations. A separate September 2026 Logistics rerun uses four of those tasks with explicit conventional signatures; simultaneous changes in prompts, operator order, model date, and parser handling prevent causal attribution of the improvement. The January archive lacks raw response text, so the worked plan pair comes only from September logs. There was no human plausibility study. Our reachability classification is relative to the exact coded schemas; repairing those schemas defines a different task. The executor's action parser and its sparse records do not allow us to separate every parsing error from every planning error. Model versions and decoding behavior may change, so later reruns cannot be mixed into these comparisons. GPT-5.2 has one temperature while the other models have two. The new follow-up covers four tasks in one domain and shows model-specific variation: Gemini 3 and Gemini 2.5 have no failures, while the 14 failures are concentrated in GPT-4.1 and GPT-5.2.

7. Conclusion and Future Work

The January audit identifies unreachable tasks, corrects the manuscript's Logistics argument order, and separates goal-reaching prefixes from fully executable returned plans. On the original Logistics

tasks, the September rerun reaches 139/140 valid final plans, so the January failure frequency is not a stable characterization of these later systems. On four new, longer Logistics tasks, the checked responses again include 13 precondition failures and an executable plan that misses a goal. Most failed preconditions concern package or truck location at `load`. Gemini 3 completed the new task grid with 40/40 independently executable final plans, bringing the cohort to 126/140. The central finding is therefore conditional on task demands: fluent planning can improve markedly on old instances while still making concrete symbolic state errors on new ones. Future evaluations should verify task solvability, retain raw final plans, and check every action against a clearly specified state and operator model.

Use of Generative AI

Generative AI tools were used during the preparation of this manuscript to assist with drafting, editing, and improving clarity of exposition. The author reviewed the technical analysis and conclusions. The author takes full responsibility for the correctness and originality of the work.

References

Aghzal, M., Plaku, E., Stein, G. J., & Yao, Z. (2025). A survey on large language models for automated planning. *arXiv preprint arXiv:2502.12435*.

Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., & Zeng, A. (2022). Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*.

Ghallab, M., Nau, D., & Traverso, P. (2004). *Automated planning: Theory and practice*. Morgan Kaufmann.

Huang, W., Abbeel, P., Pathak, D., & Mordatch, I. (2022). Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. *Proceedings of the 39th International Conference on Machine Learning* (pp. 9118–9147). PMLR.

Huang, W., et al. (2023). Inner monologue: Embodied reasoning through planning with language models. *Proceedings of the 6th Conference on Robot Learning* (pp. 1769–1782).

Kambhampati, S., Valmeekam, K., Guan, L., Verma, M., Stechly, K., Bhambri, S., Saldyt, L. P., & B Murthy, A. (2024). Position: LLMs can't plan, but can help planning in LLM-modulo frameworks. *Proceedings of the 41st International Conference on Machine Learning* (pp. 22895–22907). PMLR.

Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., & Stone, P. (2023). Llm+p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*.

McDermott, D., Ghallab, M., Howe, A., Knoblock, C., Ram, A., Veloso, M., Weld, D., & Wilkins, D. (1998). *PDDL: The planning domain definition language*. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control.

Silver, T., Dan, S., Srinivas, K., Tenenbaum, J. B., Kaelbling, L. P., & Katz, M. (2024). Generalized planning in PDDL domains with pretrained large language models. *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 20256–20264).

Song, C. H., Wu, J., Washington, C., Sadler, B. M., Chao, W.-L., & Su, Y. (2023). LLM-Planner: Few-shot grounded planning for embodied agents with large language models. *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 2998–3009).

Tantakoun, M., Muise, C., & Zhu, X. (2025). LLMs as planning formalizers: A survey for leveraging large language models to construct automated planning models. *Findings of the Association for Computational Linguistics: ACL 2025* (pp. 25167–25188).

Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., & Kambhampati, S. (2023). Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*.

Wang, Z., Cai, S., Chen, G., Liu, A., Ma, X., & Liang, Y. (2023). Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *Advances in Neural Information Processing Systems* (pp. 35221–35234).

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023a). Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023b). React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.

Zhang, T., Zhang, L., Hou, Z., Wang, Z., Gu, Y., Clark, P., Callison-Burch, C., & Tandon, N. (2024). PROC2PDDL: Open-domain planning representations from text. *Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations* (pp. 13–24).

Zheng, H. S., et al. (2024). NATURAL PLAN: Benchmarking LLMs on natural language planning. *arXiv preprint arXiv:2406.04520*.